RAPPORT D'INGÉNIERIE DES DONNÉES · PROJET 0

Ubisoft vs Electronic Arts

Ce que révèlent les données Steam :
un pipeline ETL de bout en bout, du brief client à la recette

LUMU BAMANA MANASSE

Data engineer

Date	Octobre 2026
Version	1.0.0 (livrée et recettée)
Client	Lumen Forge Games (fictif) · Mission LFG-2026-001
Dépôt GitHub	https://github.com/MediJr/ubisoft-vs-ea-steam-etl
Langue	Français · *English version available in the repository*

Scénario fictif, données réelles : le client et ses interlocuteurs sont inventés pour simuler une mission professionnelle complète. Les données analysées proviennent des API publiques de Steam et de SteamSpy, extraites le 21 octobre 2026. Ce projet n'est affilié ni à Ubisoft, ni à Electronic Arts, ni à Valve.

Fiche du document

Projet	Projet 0 du portfolio : pipeline ETL sur les données publiques de Steam
Mission	LFG-2026-001 : « Comprendre ce qui fonctionne chez les grands éditeurs sur Steam »
Client	Lumen Forge Games, studio indépendant lyonnais (fictif), à l'attention de Camille Roux
Auteur	Lumu Bamana Manasse, data engineer
Période de réalisation	Octobre 2026
Données	Extraction du 21 octobre 2026, de 18 h 38 à 19 h 10 (heure de Paris)
Statut	LIVRÉ RECETTÉ AVEC RÉSERVES MINEURES
Code source	https://github.com/MediJr/ubisoft-vs-ea-steam-etl

Historique des versions

Version	Date	Objet
0.1	21/10/2026	Cadrage : brief, compte rendu, cahier des charges
0.9	21/10/2026	Pipeline complet, analyses, corrections de qualité des données
1.0.0	22/10/2026	Recette, rapport final (français et anglais), publication

Guide de lecture

Décideur (client)	1. Résumé exécutif et 11. Résultats et recommandations
Responsable data	2. Cadrage, 4. Architecture, 7. Qualité, 10. Recette, 12. Limites
Ingénieur reprenant le projet	5. Pipeline, 6. Modèle de données, 9. Exploitation, annexes

Table des matières

Fiche du document 2

1 Résumé exécutif 4

2 Contexte et cadrage 4

2.1 De la demande floue aux questions mesurables 4
2.2 Périmètre 5
2.3 Démarche 5

3 Sources de données **6**

4 Architecture de la solution **6**
4.1 Choix techniques 7
4.2 Principes de conception 7

5 Conception du pipeline **7**
5.1 Extraction 7
5.2 Transformation 8
5.3 Chargement 8

6 Modèle de données **9**

7 Qualité des données **9**
7.1 Contrôles intégrés 9
7.2 Anomalies détectées et corrigées 10

8 Sécurité, conformité et gouvernance **11**

9 Exploitation **11**
9.1 Commandes 11
9.2 Capture d'une exécution complète 11
9.3 Incident et reprise 12

10 Tests et recette **12**

11 Résultats et recommandations **13**
11.1 Q1 · Qui satisfait le mieux ses joueurs, et durablement ? 13
11.2 Q2 · Un jeu plus cher est-il mieux noté ? 15
11.3 Q3 · Quels jeux gardent leurs joueurs ? 15
11.4 Q4 · Les francophones jugent-ils différemment ? 16
11.5 Recommandations au client 16

12 Limites et dette technique **17**

13 Retour d'expérience **17**

14 Feuille de route **18**

A Dictionnaire de données : table `games` **18**

B Décisions d'architecture (ADR) **19**

C Structure du dépôt **19**

D Glossaire **19**

1 Résumé exécutif

Le besoin. Lumen Forge Games, un studio indépendant sans équipe data, prépare la sortie de son premier jeu sur Steam à environ 25 €. Avant de fixer son prix et sa stratégie de lancement, il veut savoir ce qui fonctionne (et ce qui échoue) chez deux grands éditeurs : **Ubisoft** et **Electronic Arts**.

La solution. Un pipeline ETL en Python collecte automatiquement les données publiques de Steam et de SteamSpy (six sources), les nettoie, puis les charge dans une base analytique DuckDB. Dix requêtes SQL répondent aux quatre questions business, et un rapport présente les résultats. Le tout s'exécute en **une seule commande**.

100 jeux analysés (50 par éditeur)	**9,8 M** avis Steam agrégés	**930** réponses d'API brutes conservées	**0** donnée personnelle collectée
6 sources de données	**30 min** extraction complète (clone neuf)	**2 s** rejeu d'un lot sans appel d'API	**6 / 6** critères de recette conformes

Principaux résultats

- **Q1 · Satisfaction :** égalité en moyenne (80 %), mais deux profils opposés. EA joue « quitte ou double » (médiane de 85 %, mais 11 jeux sous 70 %). Ubisoft est plus régulier, et ses jeux **gagnent 3,2 points après leur sortie**.
- **Q2 · Prix :** plus un jeu est cher, moins il est bien noté (corrélation d'environ $-0{,}5$). Au-delà de 60 €, la satisfaction chute.
- **Q3 · Longévité :** les jeux qui durent sont gratuits ou multijoueurs.
- **Q4 · Francophones :** légèrement plus généreux (+2 points), mais ils ne rédigent que 2 % des avis.

Ce que démontre ce projet, côté ingénierie : une extraction respectueuse des API (pauses, réessais, cache de reprise), une couche brute immuable et datée, un chargement transactionnel sans doublon, la protection des données *dès la conception* (aucun pseudo de joueur n'est jamais téléchargé), la détection et la correction de quatre problèmes de qualité dans des données réelles, et une reprise après incident documentée.

2 Contexte et cadrage

2.1 De la demande floue aux questions mesurables

La demande initiale tenait en une phrase : « comprendre ce qui marche chez les gros éditeurs ». La réunion de cadrage l'a transformée en quatre questions, chacune liée à une décision

concrète du client.

#	Question business	Décision éclairée
Q1	Lequel des deux éditeurs satisfait le mieux ses joueurs sur Steam, et cette satisfaction est-elle stable dans le temps ?	Suivi de réputation après la sortie
Q2	Un jeu plus cher est-il mieux noté ? Les promotions sont-elles un signe de difficulté ?	Prix de lancement et promotions
Q3	Quels jeux gardent leurs joueurs des années après leur sortie ?	Contenu après le lancement
Q4	Les joueurs francophones jugent-ils différemment ces éditeurs ?	Marché prioritaire

TABLE 1 – Questions business retenues lors du cadrage

2.2 Périmètre

Inclus	Exclu
Jeux de type `game` publiés sur Steam par Ubisoft ou EA	DLC, bandes-son, démos, éditions groupées
Au moins 500 avis, 50 jeux maximum par éditeur (les plus commentés)	Autres plateformes : EA app, Ubisoft Connect, consoles
Fiches, synthèses d'avis, historique mensuel, fréquentation	Texte des avis et toute donnée personnelle
Rapport d'analyse et pipeline relançable	Impact boursier, tableau de bord interactif, planification automatique

2.3 Démarche

Le projet suit les cinq phases d'une mission de conseil en données. Chaque phase produit des livrables versionnés dans le dépôt.

Phase	Objet	Livrables
1	Cadrage	Brief client, compte rendu de cadrage, cahier des charges (`docs/mission/`)
2	Extraction	`src/extract.py`, couche brute `data/raw/`, ADR 002
3	Transformation et chargement	`src/transform.py`, `src/load.py`, `sql/00_schema.sql`
4	Analyse	10 requêtes SQL, 5 graphiques, résultats générés automatiquement
5	Livraison	Recette (PV), présent rapport, documentation, publication

TABLE 2 – Phases de la mission

3 Sources de données

Source	Statut	Données utilisées	Usage
Recherche Steam Store	Non officiel	Liste des jeux d'un éditeur (identifiants `app_id`)	Périmètre
Fiche Steam (`appdetails`)	Public, non documenté	Éditeur, prix en euros, remise, genres, date de sortie	Q2, Q3
Avis (`appreviews`)	Public, documenté	Totaux d'avis, toutes langues et en français	Q1, Q4
Historique (`appreviewhistogram`)	Non officiel	Avis positifs et négatifs par mois	Q1
Steam Web API	Officiel	Joueurs connectés	Q3
SteamSpy	Tiers	Estimation des possesseurs, pic de la veille	Information

TABLE 3 – Inventaire des sources

Contraintes. Steam tolère environ 200 requêtes par tranche de 5 minutes. Trois points d'accès ne sont pas officiels et peuvent changer sans préavis. Aucune clé d'API n'est requise, donc il n'y a aucun secret à gérer.

> **Exploration préalable : tester avant de s'engager**
>
> Avant de signer le cahier des charges, chaque point d'accès a été testé à la main. Le paramètre `day_range` de l'API des avis *semble* filtrer les avis sur une période, mais en réalité il **ne modifie pas** la synthèse renvoyée. Sans ce test, la mesure de stabilité (Q1) aurait été promise sur une base fausse. C'est l'historique mensuel (`appreviewhistogram`) qui l'a rendue possible.

4 Architecture de la solution

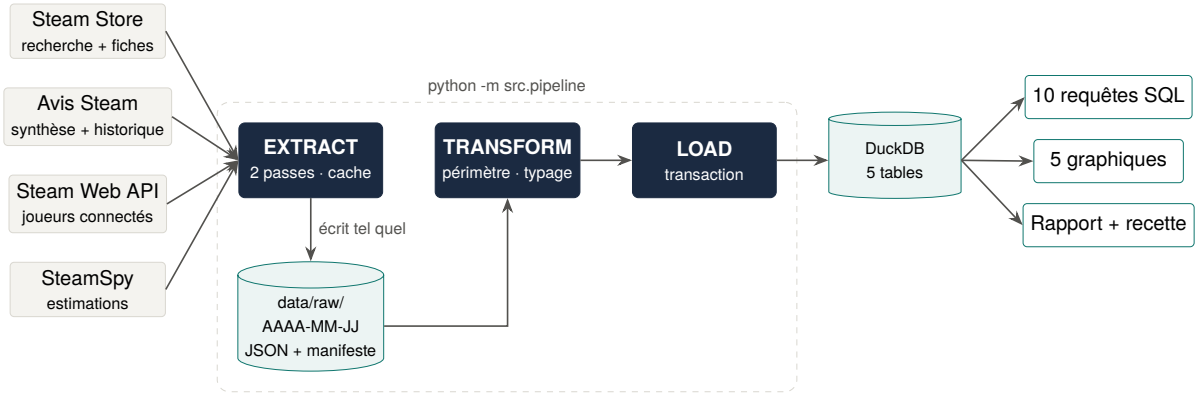


FIGURE 1 – Architecture du pipeline : une commande enchaîne extraction, transformation, chargement et production du rapport

4.1 Choix techniques

Besoin	Choix	Justification et alternatives écartées
Langage	Python 3.14	Standard du data engineering, écosystème riche
Appels HTTP	`requests`	Simple et fiable. `httpx` (asynchrone) est inutile à ce volume.
Transformation	`pandas`	Adapté à quelques milliers de lignes. Polars est plus rapide, mais sans gain ici.
Stockage	DuckDB (fichier)	SQL analytique sans serveur. PostgreSQL sera la nouveauté du projet 1.
Analyse	SQL	Lisible, vérifiable, indépendant de Python
Graphiques	`matplotlib`	Figures statiques intégrables au rapport
Environnement	`venv` + `pip`	Universel. `uv` et Poetry seront introduits plus tard.
Orchestration	Aucune (une commande)	Volontaire au niveau 0. Dagster est prévu au projet 3.

TABLE 4 – Stack technique (détail : ADR 001)

4.2 Principes de conception

1. **Couche brute immuable :** les réponses des API sont stockées sans modification. Un bug de transformation se corrige en rejouant le lot, sans réinterroger les API.
2. **Idempotence :** exécuter deux fois le pipeline donne le même résultat, sans doublon (clés primaires, rechargement complet).
3. **Protection des données dès la conception :** la donnée personnelle n'est jamais demandée aux API.
4. **Reproductibilité :** chaque chiffre du rapport provient d'une requête versionnée. Le fichier `resultats.md` est régénéré à chaque exécution.
5. **Traçabilité :** un manifeste par lot, une table des exclusions motivées, et trois ADR pour les décisions structurantes.

5 Conception du pipeline

5.1 Extraction

Collecter les six sources pour les 160 jeux candidats aurait demandé environ 1 000 appels, soit plus de 25 minutes, pour des jeux dont une partie serait de toute façon écartée. L'extraction se fait donc en **deux passes** :

Étape	Action	Ubisoft	EA
Découverte	Recherche Steam par éditeur, catégorie « jeux »	80	80
Passe légère	Synthèse des avis (1 appel par jeu), filtre « au moins 500 avis »	77	76
Passe détaillée	5 appels par jeu retenu	77	76

Table 5 – Entonnoir d'extraction (lot du 21/10/2026)

Robustesse. Une pause de 1,5 s précède chaque appel. Les erreurs 429 (trop de requêtes) et 5xx déclenchent jusqu'à trois réessais à délai croissant. Un `User-Agent` explicite identifie le projet. Chaque réponse est mise en cache dans la couche brute : une extraction interrompue reprend là où elle s'est arrêtée. L'écriture est **atomique**, ajoutée après l'incident décrit en section 9.3 :

```
temporary = path.with_suffix(".tmp")
temporary.write_text(json.dumps(payload, ensure_ascii=False, indent=2), encoding="utf-8")
temporary.replace(path)   # renommage instantané : jamais de fichier à moitié écrit
```

Protection des données dès la source. Les avis sont demandés avec `num_per_page=0` : Steam ne renvoie que des totaux, sans aucun avis individuel, donc sans pseudo ni identifiant.

5.2 Transformation

Règle	Mise en œuvre
Périmètre	Type `game`, déjà sorti, éditeur réel conforme à l'éditeur recherché
Plafond	Les 50 jeux les plus commentés de chaque éditeur. Les autres sont tracés dans `exclusions` (53 jeux).
Typage	Dates françaises (« 12 janv. 2023 ») converties en `DATE`, prix en euros, fourchettes SteamSpy en entiers
Valeurs manquantes	Taux d'avis `NULL` plutôt que 0 en l'absence d'avis. Un jeu gratuit vaut 0 €.
Historique	Horodatages Unix convertis en mois, agrégés si Steam renvoie un découpage hebdomadaire

Table 6 – Règles de transformation

5.3 Chargement

Le chargement suit une stratégie de **rechargement complet** : le schéma est recréé, puis toutes les tables sont insérées, **dans une seule transaction**. En cas d'échec, rien n'est validé et la base garde son état précédent. Elle n'est jamais à moitié chargée.

```
connection.execute("BEGIN TRANSACTION")
try:
    connection.execute(SCHEMA_PATH.read_text(encoding="utf-8"))
    for name in LOAD_ORDER:                    # games d'abord (clés étrangères)
        connection.register("incoming", tables[name])
        connection.execute(f"INSERT INTO {name} BY NAME SELECT * FROM incoming")
        connection.unregister("incoming")
```

```
    connection.execute("COMMIT")
except Exception:
    connection.execute("ROLLBACK")
    raise
```

6 Modèle de données



FIGURE 2 – Modèle de données DuckDB (les flèches représentent les clés étrangères)

Table	Grain (une ligne par...)	Lignes
`games`	jeu retenu	100
`review_summary`	jeu et langue (`all`, `french`)	200
`reviews_monthly`	jeu et mois	9 929
`player_snapshot`	jeu et date d'extraction	100
`exclusions`	jeu candidat écarté, avec son motif	53

TABLE 7 – Tables, grain et volumétrie (dictionnaire complet en annexe A)

7 Qualité des données

7.1 Contrôles intégrés

— **Contraintes du schéma :** clés primaires, `NOT NULL`, clés étrangères. Une anomalie fait échouer le chargement au lieu de fausser les résultats.

— **Requête de contrôle** (`sql/controle_recette.sql`) : absence de doublons, volume par éditeur, seuil d'avis, absence de colonne personnelle.

— **Table des exclusions :** chaque candidat écarté a un motif. Rien ne disparaît sans trace.

7.2 Anomalies détectées et corrigées

Les données réelles ont révélé quatre problèmes que la seule lecture de la documentation des API ne laissait pas prévoir.

Symptôme	Cause	Correction
5 dates de sortie non reconnues	Steam écrit « aout » sans accent, et une fiche n'a aucune date	Dictionnaire des mois accepte les deux graphies. La date absente reste `NULL`.
0 exclusion au premier passage : un résultat trop parfait	Les jeux écartés par le plafond de 50 n'étaient pas tracés	53 exclusions motivées enregistrées
Pic de joueurs à 0 pour 9 jeux récents (*Battlefield 6* : 0, contre 45 100 connectés)	SteamSpy ne suit pas encore ces jeux : 0 signifie « inconnu »	Bascule sur l'API officielle de Steam (ADR 003)
« Lancement » de 21 jeux EA en 2020	Retour d'EA sur Steam en 2020, et avis plus récents que certains jeux	Q1b exige des premiers avis contemporains de la sortie sur Steam

TABLE 8 – Problèmes de qualité rencontrés sur les données réelles

> **Leçon**
>
> Un zéro n'est pas toujours un zéro, et un résultat parfait est suspect. La qualité des données ne se décrète pas : elle se vérifie, jeu par jeu, avant toute analyse.

8 Sécurité, conformité et gouvernance

Thème	Mesure
Données personnelles (RGPD)	L'API des avis peut renvoyer pseudos, identifiants et textes. En application de la **minimisation** (art. 5.1.c) et de la **protection dès la conception** (art. 25), seuls des totaux sont demandés. Vérification : aucun champ `steamid` ni `personaname` dans les 930 fichiers bruts, et aucune colonne personnelle dans la base.
Secrets	Aucune clé d'API requise. Le `.gitignore` exclut néanmoins tout fichier `.env`.
Données publiées	Les données ne sont pas versionnées (`data/` est ignoré) : elles se régénèrent avec le pipeline.
Usage des API	Débit limité volontairement, `User-Agent` explicite, usage non commercial
Traçabilité	Manifeste par lot, journal de bord daté, ADR, CHANGELOG

9 Exploitation

9.1 Commandes

```
python -m venv .venv && .venv\Scripts\activate     # environnement isolé (Windows)
pip install -r requirements.txt
python -m src.pipeline                             # extraction du jour + traitement (~30 min)
python -m src.pipeline --run-date 2026-10-21       # rejeu d'un lot brut, sans appel d'API (~2 s)
```

9.2 Capture d'une exécution complète

Journal réel de la recette sur un environnement neuf (horodatages et chemins abrégés, progression intermédiaire masquée) :

PowerShell · python -m src.pipeline

```
19:16:24 | INFO | src.extract   | Extraction vers ...\00-jeux-video-etl\data\raw\2026-10-21
19:16:29 | INFO | src.extract   | Ubisoft : 80 jeux candidats
19:19:17 | INFO | src.extract   | Ubisoft : 76 jeux avec au moins 500 avis
19:31:55 | INFO | src.extract   | Electronic Arts : 80 jeux candidats
19:34:39 | INFO | src.extract   | Electronic Arts : 76 jeux avec au moins 500 avis
19:46:47 | INFO | src.extract   | Extraction terminée
19:46:47 | INFO | src.transform | Table games            :    100 lignes
19:46:47 | INFO | src.transform | Table review_summary   :    200 lignes
19:46:47 | INFO | src.transform | Table reviews_monthly  :   9768 lignes
19:46:47 | INFO | src.transform | Table player_snapshot  :    100 lignes
19:46:47 | INFO | src.transform | Table exclusions       :     52 lignes
19:46:47 | INFO | src.load      | Base chargée : ...\data\steam.duckdb
19:46:49 | INFO | src.report    | Graphiques et résultats écrits dans ...\docs\livraison
19:46:49 | INFO | pipeline      | Pipeline terminé en 1825 s
```

9.3 Incident et reprise

Heure	Événement
18 h 38	Lancement de l'extraction de production
19 h 05	**Interruption externe** du processus, à 56 jeux EA sur 76
19 h 06	Diagnostic : aucun processus actif, aucun manifeste, 133 jeux en cache. Vérification : aucun fichier JSON tronqué.
19 h 06	Relance de la même commande : les 133 jeux en cache sont relus en **0,1 s**, puis la collecte reprend
19 h 10	Extraction terminée. Amélioration : écriture atomique, pour qu'un arrêt brutal ne laisse jamais de fichier corrompu.

TABLE 9 – Chronologie de l'incident du 21/10/2026

10 Tests et recette

La recette a été menée sur un **environnement neuf** : copie des seuls fichiers publiables (38 fichiers, sans données ni environnement virtuel), nouvel environnement Python, puis la commande unique du README.

#	Critère	Résultat	
C1	Exécution de bout en bout en une commande	Installation en 54 s, pipeline en 30 min 25 s, code de retour 0	✓
C2	Aucun doublon après une nouvelle exécution	Volumes identiques avant et après : 100 / 200 / 9 929 / 100 / 53	✓
C3	Au moins 15 jeux par éditeur	50 jeux par éditeur, tous avec au moins 500 avis (minimum : 7 215)	✓
C4	Une requête et une réponse par question	10 requêtes, une section par question	✓
C5	Aucune donnée personnelle	Aucun champ personnel dans 930 fichiers bruts ni dans la base	✓
C6	Limites explicitées	Section 12	✓

TABLE 10 – Résultats de la recette (détails : `docs/livraison/2026-10-21-pv-recette.md`)

Réserves mineures

R1 · Reproductibilité du panel. L'ordre de pertinence de la recherche Steam varie légèrement d'une exécution à l'autre. Entre la production et la recette, 99 jeux sur 100 sont identiques. Les indicateurs clés le sont au dixième près (taux pondérés : EA 77,4 %, Ubisoft 81,3 %).
R2 · Promotions. Aucun jeu n'était en promotion le jour de l'extraction : cette partie de Q2 ne peut pas être tranchée.

11 Résultats et recommandations

11.1 Q1 · Qui satisfait le mieux ses joueurs, et durablement ?

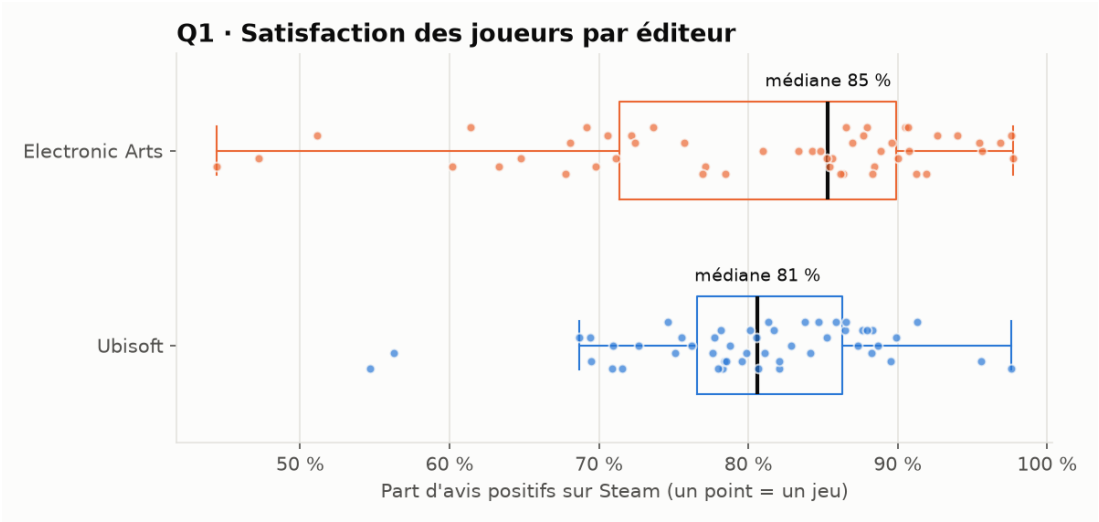


FIGURE 3 – Part d'avis positifs par jeu et par éditeur

Éditeur	Moyenne	Médiane	Pondéré	< 70 %	Évolution	Redressés
Electronic Arts	80,3 %	85,3 %	77,4 %	11	−1,0 pt	4 / 34
Ubisoft	80,2 %	80,6 %	81,3 %	5	+3,2 pts	12 / 39

TABLE 11 – Satisfaction globale, puis évolution entre les 3 premiers mois et les 12 derniers mois sur Steam

Égalité en moyenne, mais deux profils : EA est plus dispersé (meilleure médiane, mais deux fois plus de jeux sous 70 %), tandis qu'Ubisoft est plus régulier et **améliore ses jeux après leur sortie** (*Assassin's Creed Unity* +24,5 points, *The Crew 2* +25,5).

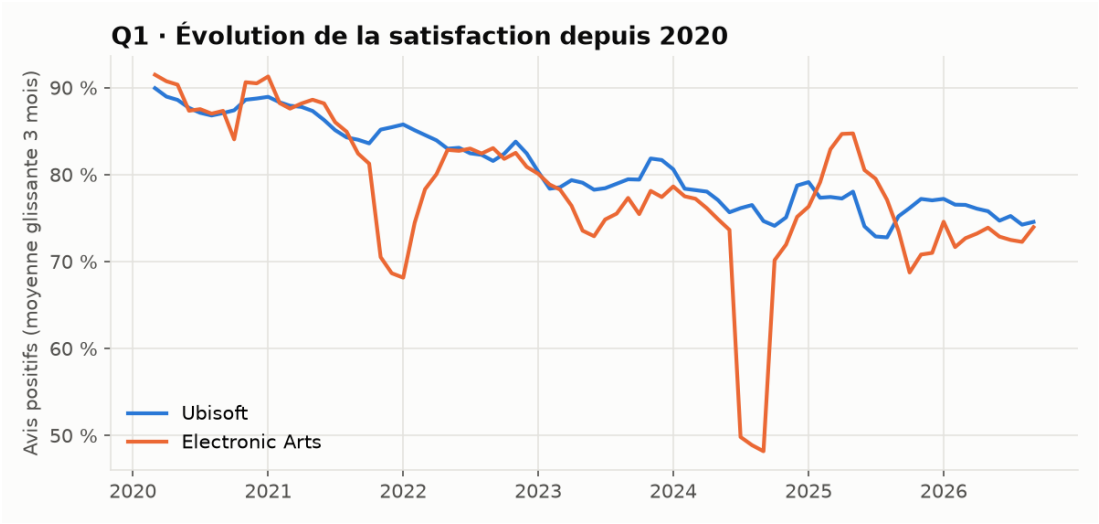


FIGURE 4 – Évolution mensuelle : deux crises visibles chez EA

L'historique mensuel révèle deux crises chez EA : le lancement de *Battlefield 2042* (novembre 2021 : 40 065 avis négatifs, 35 % de positifs, redressé depuis de +30,7 points), et un *review bombing* d'*Apex Legends* en juillet 2024 (82 120 avis négatifs en un mois, soit 7 % de positifs), six ans après sa sortie.

11.2 Q2 · Un jeu plus cher est-il mieux noté ?

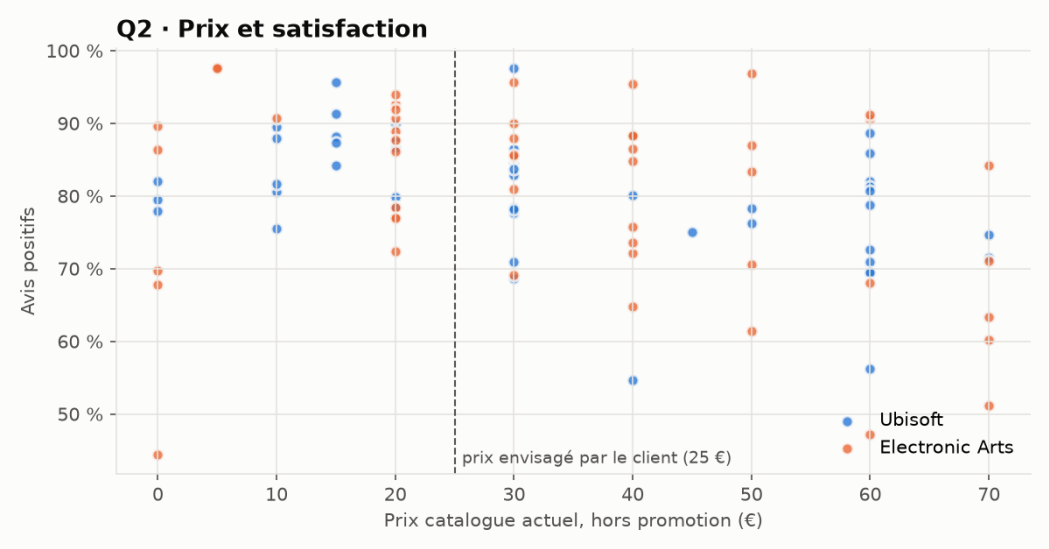


FIGURE 5 – Prix catalogue actuel et satisfaction

Non, c'est plutôt l'inverse : la corrélation vaut $-0{,}55$ chez EA et $-0{,}46$ chez Ubisoft (Spearman : $-0{,}49$ pour les deux). À 60 € et plus, la médiane tombe à 63 % (EA) et 73 % (Ubisoft). La tranche 20 à 40 €, celle du client, se situe entre 80 et 86 %. Le prix étant le prix catalogue *actuel*, un **biais du survivant** est possible : les anciens jeux peu chers qui se vendent encore sont souvent des classiques.

11.3 Q3 · Quels jeux gardent leurs joueurs ?

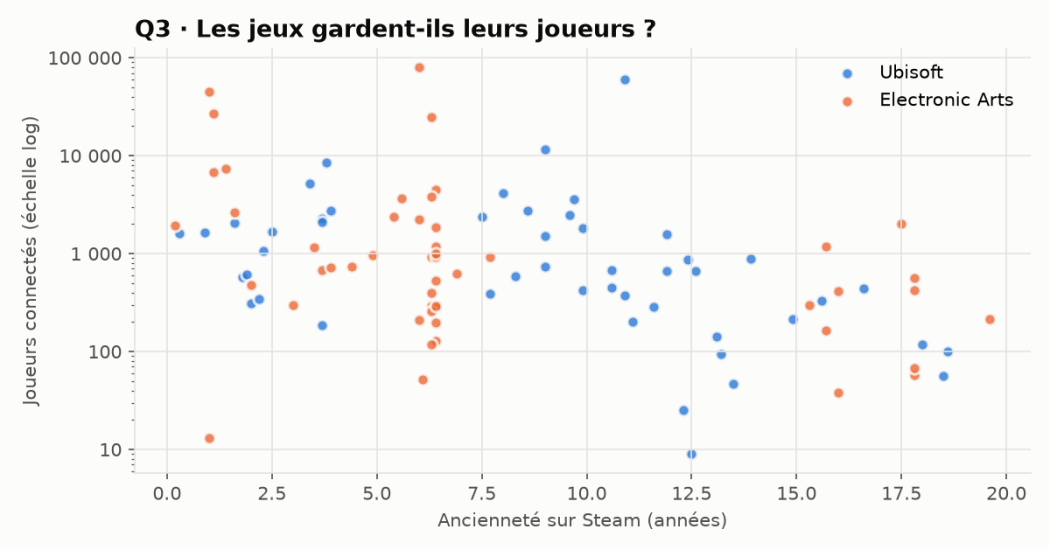


FIGURE 6 – Ancienneté sur Steam et joueurs connectés (échelle logarithmique)

Les quatre jeux de 5 ans et plus les plus joués sont **gratuits** : *Apex Legends* (79 893 joueurs connectés), *Rainbow Six Siege* (60 193), *Les Sims 4* et *Brawlhalla*. Parmi les jeux payants, ce sont le multijoueur, la coopération (*It Takes Two*, 95 % d'avis positifs) et les grands jeux de rôle qui durent.

11.4 Q4 · Les francophones jugent-ils différemment ?

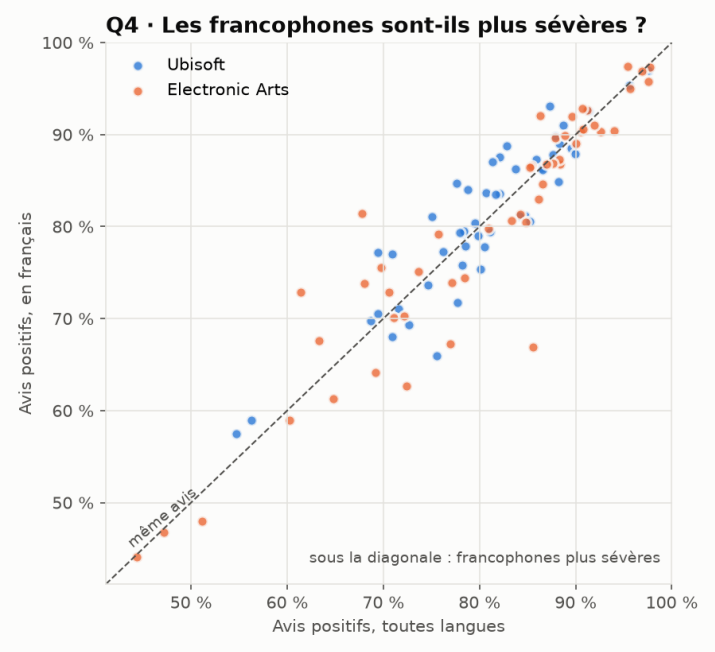


Figure 7 – Satisfaction en français et toutes langues, par jeu

Les francophones sont légèrement plus généreux, de +2,5 points envers EA et +2,1 envers Ubisoft. Sur 9 jeux Ubisoft ils sont nettement plus positifs, contre 2 où ils sont plus sévères. Mais ils ne rédigent que **2 % des avis**.

11.5 Recommandations au client

Pour Lumen Forge Games

1. **Prix :** 25 € se situe dans une tranche bien notée. Monter en prix, c'est monter les attentes.
2. **Le lancement n'est pas un verdict :** prévoyez un calendrier de correctifs. Une réputation se redresse.
3. **Surveillez les avis mois par mois :** une crise peut survenir des années après la sortie.
4. **Durée de vie :** des mises à jour de contenu ou un mode coopératif prolongent la vie d'un jeu.
5. **Marché :** traduisez le jeu en français, mais visez l'international, qui fait la réputation

> Steam.

12 Limites et dette technique

Limite	Conséquence
Steam uniquement	Les conclusions portent sur les joueurs Steam, sans les autres plateformes
Instantané (mercredi, vers 19 h)	Pas de saisonnalité. Promotions non mesurables.
Prix catalogue actuel	Biais du survivant possible (Q2)
Ancienneté sur Steam	Différente de l'ancienneté réelle pour les jeux réédités (Q1b, Q3)
« Francophones » = avis en français	Ni le pays ni la langue maternelle ne sont connus
Corrélation	Elle ne prouve pas de causalité
Dette technique assumée au niveau 0	
Pas de tests unitaires	Introduits au projet 1 (`pytest`)
Pas de planification	Exécution manuelle. Orchestration prévue au projet 3.
Découverte non déterministe	Réserve R1. Parcourir tout le catalogue de l'éditeur.
Rechargement complet	Pas d'historique entre deux exécutions. Chargement incrémental prévu au projet 2.

13 Retour d'expérience

— **Cadrer avant de coder :** une demande floue est devenue quatre questions mesurables, chacune liée à une décision du client.

— **Tester les API avant de promettre :** le paramètre `day_range` ne fait pas ce que son nom suggère.

— **Se méfier des résultats parfaits :** « 0 exclusion » cachait un défaut de traçabilité.

— **Concevoir pour l'incident :** grâce au cache, une interruption s'est réglée en une minute.

— **Dire ce que les données ne permettent pas de conclure :** la question des promotions reste ouverte, et le rapport le dit.

14 Feuille de route

Évolution	Apport
Tests et PostgreSQL	Robustesse et base serveur (projet 1 du portfolio)
Chargement incrémental quotidien	Historique des prix, saisonnalité, promotions (projet 2)
Orchestration et dbt	Planification, couches *bronze, silver, gold* (projet 3)
Alerte de réputation	Notification quand la part d'avis positifs d'un mois passe sous un seuil
Panel d'éditeurs indépendants	Comparaison plus proche de la taille du client

A Dictionnaire de données : table `games`

Colonne	Type	Description
`app_id`	INTEGER	Identifiant Steam du jeu (clé primaire)
`name`	VARCHAR	Nom du jeu
`publisher_group`	VARCHAR	Éditeur analysé : Ubisoft ou Electronic Arts
`publishers` / `developers`	VARCHAR	Éditeur(s) et studio(s) déclarés sur Steam
`is_free`	BOOLEAN	Jeu gratuit (*free-to-play*)
`price_initial_eur`	DECIMAL	Prix catalogue actuel hors promotion, en euros (et non le prix de lancement)
`price_final_eur`	DECIMAL	Prix payé le jour de l'extraction
`discount_percent`	INTEGER	Remise en cours (%)
`steam_release_date`	DATE	Date de sortie sur Steam
`years_on_steam`	DECIMAL	Ancienneté sur Steam au jour de l'extraction
`genres`	VARCHAR	Genres Steam, en français
`metacritic_score`	INTEGER	Note de la presse, si disponible
`owners_estimate_low/high`	BIGINT	Fourchette SteamSpy du nombre de possesseurs
`extracted_on`	DATE	Date de l'extraction

Dictionnaire complet des cinq tables : `docs/dictionnaire-de-donnees.md`.

B Décisions d'architecture (ADR)

ADR 001	Stack technique : DuckDB, pandas, `requests`, rechargement complet
ADR 002	Stratégie d'extraction : deux passes, couche brute datée, cache, minimisation à la source
ADR 003	Source de fréquentation : API officielle de Steam plutôt que SteamSpy

C Structure du dépôt

```
ubisoft-vs-ea-steam-etl/
|-- src/          extract.py, transform.py, load.py, report.py, pipeline.py, config.py
|-- sql/          00_schema.sql, q1a ... q4b (10 requetes), controle_recette.sql
|-- docs/
|   |-- mission/      brief, compte rendu de cadrage, cahier des charges
|   |-- decisions/    ADR 001 a 003
|   |-- journal/      journal de bord date
|   |-- livraison/    rapports PDF, resultats.md, figures, PV de recette
|   `-- rapport/      sources LaTeX de ce rapport
|-- data/         lots bruts et base DuckDB (non versionnes, regeneres par le pipeline)
|-- README.md  CHANGELOG.md  LICENSE  requirements.txt
```

D Glossaire

ETL	*Extract, Transform, Load* : extraire, transformer, charger
Couche brute	Données conservées telles que reçues des sources, sans modification
Idempotence	Propriété d'un traitement qui donne le même résultat, qu'on l'exécute une ou plusieurs fois
Rechargement complet	Remplacement intégral des tables à chaque exécution (*full refresh*)
Écriture atomique	Écriture qui réussit entièrement ou pas du tout
ADR	*Architecture Decision Record* : fiche qui trace une décision technique et ses raisons
Recette	Vérification formelle, par le client, que le livrable respecte les critères convenus
Review bombing	Afflux massif et soudain d'avis négatifs, souvent en réaction à une décision de l'éditeur